

GARGI WADEKAR ROLL NO.23

```

import numpy as np

# Activation functions
def relu(x):
    return np.maximum(0, x)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# ----- User Input -----
print("Enter bike details for price prediction:")
cc = int(input("Engine CC: "))
mileage = int(input("Mileage (km/l): "))
abs_feature = int(input("ABS (1 = Yes, 0 = No): "))

# Input vector
X = np.array([cc, mileage, abs_feature])
print("\nInput Vector:", X.tolist())

# ----- Hidden Layer 1 -----
W1 = np.array([
    [0.02, 0.01, 0.5],
    [0.03, 0.04, 0.2],
    [0.05, 0.02, 0.3],
    [0.01, 0.03, 0.4]
])
b1 = np.array([1, 1, 1, 1])

H1 = relu(np.dot(W1, X) + b1)

print("\n--- Hidden Layer 1 Output (ReLU) ---")
for i, val in enumerate(H1):
    print(f"Neuron {i+1}: {val:.4f}")

# ----- Hidden Layer 2 -----
W2 = np.array([
    [0.2, 0.3, 0.1, 0.4],
    [0.5, 0.2, 0.3, 0.1],
    [0.1, 0.4, 0.2, 0.5]
])
b2 = np.array([1, 1, 1])

H2 = relu(np.dot(W2, H1) + b2)

print("\n--- Hidden Layer 2 Output (ReLU) ---")
for i, val in enumerate(H2):
    print(f"Neuron {i+1}: {val:.4f}")

# ----- Output Layer -----
W3 = np.array([0.4, 0.3, 0.2])
b3 = 1

output = sigmoid(np.dot(W3, H2) + b3)

print("\n--- Output Layer ---")
print(f"Sigmoid Output: {output:.4f}")

# Final decision
if output >= 0.7:
    print("Predicted Bike Price: HIGH 🚀")
elif output >= 0.4:
    print("Predicted Bike Price: MEDIUM 🌱")
else:
    print("Predicted Bike Price: LOW 🟡")

```

```

Enter bike details for price prediction:
Engine CC: 200
Mileage (km/l): 45
ABS (1 = Yes, 0 = No): 1

```

```
Input Vector: [200, 45, 1]
```

```

---- Hidden Layer 1 Output (ReLU) ----
Neuron 1: 5.9500
Neuron 2: 9.0000
Neuron 3: 12.2000
Neuron 4: 4.7500

```

---- Hidden Layer 2 Output (ReLU) ----

Neuron 1: 8.0100

Neuron 2: 9.9100

Neuron 3: 10.0100

---- Output Layer ----

Sigmoid Output: 0.9999

Predicted Bike Price: HIGH 🚲